



Practical Labs

Labs 09 · 10 · 11

MongoDB CRUD · Aggregation · REST APIs

Lab 09	Lab 10	Lab 11
CRUD Operations	Aggregation	REST APIs

Lab 09 MongoDB CRUD Operations

Objective: Manipulate NoSQL databases using MongoDB Compass.

□ Tool: MongoDB Compass · Database: storeDB · Collection: products

01 Create a products collection

1. Open MongoDB Compass → connect to localhost:27017
2. Click + next to Databases → set name to storeDB
3. Set collection name to products → click Create Database

02 Insert five documents

1. Select the products collection
2. Click ADD DATA → Insert Document → switch to JSON array view
3. Write a JSON array of 5 product objects — include at least: name, category, price, stock
4. Make sure at least one product has price > 1000
5. Click Insert — verify 5 documents appear

□ Each document automatically receives a unique `_id` field generated by MongoDB.

03 Display products with price > 1000

1. Go to the Documents tab of the products collection
2. In the FILTER bar, write a query using the correct comparison operator
3. Press Find — only products above 1000 should appear

□ Syntax reminder: { "field": { "\$operator": value } } → operators: \$gt \$lt \$gte \$lte

04 Update one product

1. Locate any product in the list
2. Hover over it → click the pencil (edit) icon
3. Modify the price and/or stock values
4. Click Update to save

□ Update operator reference: \$set modifies a field · \$inc increments a numeric field

Lab 10 MongoDB Aggregation

Objective: Build aggregation pipelines to compute and group data.

□ Tool: Aggregations tab in MongoDB Compass · Same products collection as Lab 09

How to use the Aggregations tab:

1. Select the products collection → click the Aggregations tab
2. Click Add Stage → choose the operator from the dropdown
3. Fill in the stage configuration — results preview updates live

01 Compute the average product price

1. Add a \$group stage
2. Set _id to null (group all documents together)
3. Add a computed field using the \$avg accumulator on the price field
4. Check that the result shows one document with the average

□ Field reference: always prefix with \$ inside pipeline stages → "\$price" not "price"

02 Group products by category

1. Add a \$group stage
2. Set _id to the category field (use the \$ prefix)
3. Add a counter field using \$sum: 1
4. Add an average price per category using \$avg
5. Verify: one result document per distinct category

03 Sort products by descending price

1. Add a \$sort stage
2. Set the price field to -1 (descending order)
3. Verify: most expensive product appears first
4. Bonus — add a second sort criterion to sort alphabetically within same price

□ Sort values: 1 = ascending (low → high) · -1 = descending (high → low)

Lab 11 REST APIs

Objective: Consume a public REST API and display data dynamically.

□ Tool: any text editor + a web browser · Deliverable: a single HTML file

Background — What is a REST API?

A REST API (Representational State Transfer) exposes data over HTTP. You send a request to a URL (endpoint) and receive a JSON response. No page reload is needed — JavaScript fetches the data asynchronously.

HTTP Method	Action
GET	Retrieve data from the server
POST	Send new data to the server
PUT	Replace an existing resource
DELETE	Remove a resource

Key concepts you will use:

Concept / Syntax	Description
<code>fetch(url)</code>	JavaScript function — sends an HTTP GET request
<code>.then(r => r.json())</code>	Parses the JSON response into a JavaScript object
<code>.then(data => ...)</code>	Callback — receives the parsed data to display
<code>innerHTML</code>	HTML property — lets you inject HTML into a DOM element
<code>async / await</code>	Modern syntax — cleaner alternative to <code>.then()</code> chaining

Standard fetch pattern:

```
fetch('https://api.example.com/endpoint')
  .then(response => response.json())
  .then(data => {
    // use data here - update the page
    document.getElementById('output').innerHTML = data.someField;
  })
  .catch(error => console.error('Error:', error));
```

Suggested public APIs (no authentication required):

API	Endpoint	Content
JSONPlaceholder	https://jsonplaceholder.typicode.com/posts	Fake posts, users, todos
Open-Meteo	https://api.open-meteo.com/v1/forecast?...	Weather data
REST Countries	https://restcountries.com/v3.1/all	Country info, flags

Dog CEO	https://dog.ceo/api/breeds/image/random	Random dog image URL
CoinGecko	https://api.coingecko.com/api/v3/coins/list	Cryptocurrency list

01 Create an HTML/JavaScript page that consumes a public API

1. Create a file named lab11.html
2. Write a basic HTML page: a title, a button, and an empty container div
3. Add a `<script>` block with a function triggered by the button click
4. Inside the function, use `fetch()` to call a public API of your choice
5. Parse the JSON response and inject the data into your container div
6. Display at least 3 fields from the API response (e.g. title, body, id)
7. Test in your browser — data must appear without any page reload

□ Start with JSONPlaceholder: `fetch('https://jsonplaceholder.typicode.com/posts/1')` — returns a single post object.

□ Open the browser DevTools (F12) → Console tab to see fetch errors and debug.

Minimum HTML structure:

```
<!DOCTYPE html>
<html>
<head>
  <title>Lab 11 - REST API</title>
</head>
<body>
  <h1>API Data</h1>
  <button onclick="loadData()">Load Data</button>
  <div id="result"></div>

  <script>
    function loadData() {
      // your fetch() call goes here
    }
  </script>
</body>
</html>
```

□ Expected result: clicking the button fetches data from the API and displays it inside the page — no reload.